Beyond the MLP — CNNs & Transformers

Machine Learning with Python — Lecture 2, Session 3 (Oct 20)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

Beyond the MLP — architectures that exploit the *structure* of data

- **From vectors to structure** — why an MLP is the wrong tool for pixels and sequences
- **Convolutional networks** — locality, weight sharing, pooling
- **Transformers** — self-attention, multi-head attention and positional encoding
- **Autoencoders** — representation learning without labels

and when to reach for **which** model.

Session 3 of 4 today.

Part 1 — Beyond the MLP

From vectors to structure

An MLP treats its input as a **flat vector** — every neuron sees **every** input. For structured data that is a poor fit:

- **Images.** A 256×256 RGB image is 196,608 numbers; one dense layer of 1,000 units already has **~200 million** weights — and it must *re-learn* from scratch that a cat is a cat one pixel to the left.
- **Sequences.** Text and time series are **variable length** and full of **long-range** dependencies; a fixed-size vector cannot express order.

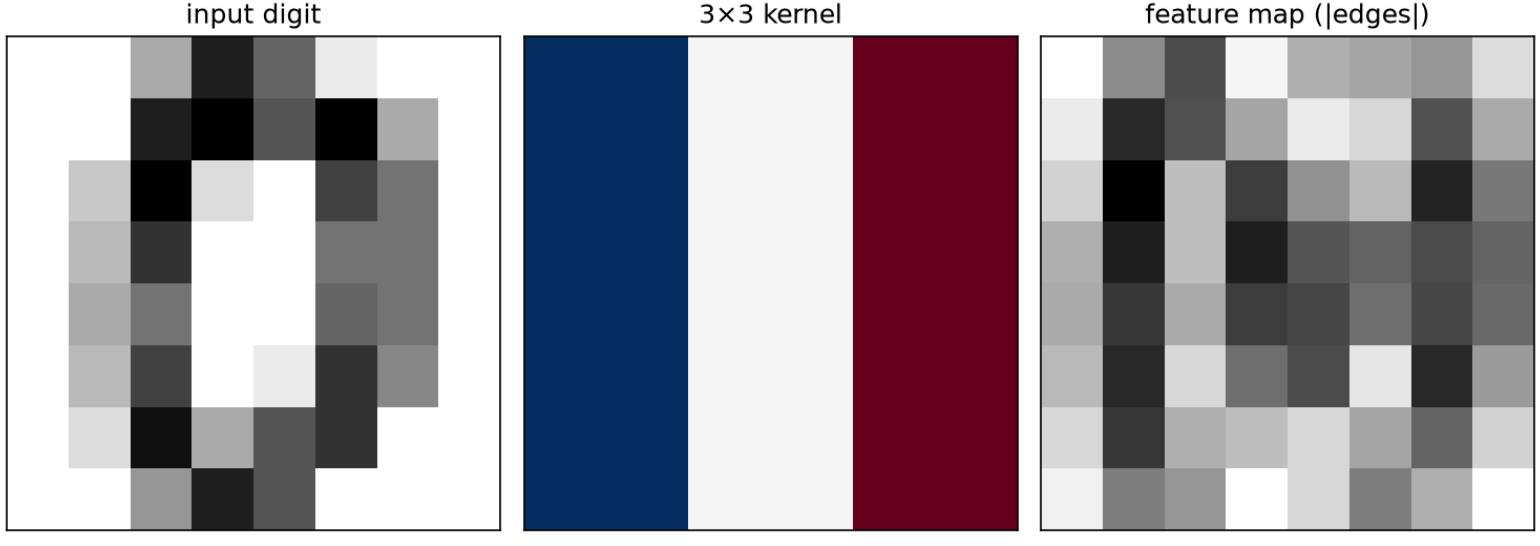
The fix is to build the right **inductive bias** into the architecture: assume what is true of the data — that pixels are **local**, that objects are **translation-invariant**, that tokens are **relational**. Two architectures dominate: **CNNs** and **transformers**.

Convolutional neural networks

A **convolution** slides a small **kernel** (say 3×3) across the image and computes a weighted sum at every position — a **feature map**:

- **Locality** — each output looks at a small neighbourhood, just like a neuron, but the *same* kernel is reused everywhere.
- **Weight sharing / translation invariance** — one kernel detects a feature wherever it appears in the image.
- **Few parameters** — a 3×3 kernel has 9 weights, no matter the image size.

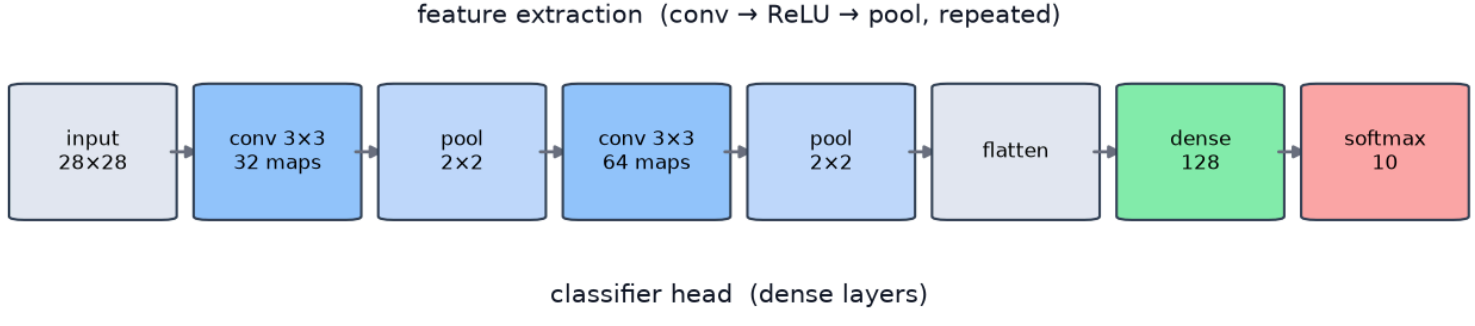
Stack convolutions and you get a **feature hierarchy**: early layers find **edges**, deeper layers compose them into **textures, parts and objects**. Below: a digit, an edge-detecting kernel, and its feature map.


input digit
3×3 kernel
feature map (|edges|)

A CNN architecture

A typical CNN repeats one **block** — **conv → ReLU → pool** — and ends with a small **classifier head**:

- **Pooling** (e.g. 2×2 max) **downsamples** the feature maps: the spatial size shrinks, the **receptive field** grows (how much of the image each unit "sees"), and a little translation tolerance is added.
- After a few blocks the maps are **flattened** and fed to dense layers — from there it is an ordinary MLP.



Training a CNN

Nothing about the *learning* changes: it is still **backpropagation** and **gradient descent**. The kernels are **learned** — we never hand-write an edge detector; the network discovers whatever filters minimise the loss.

This is the punchline of the whole lecture: in Lecture 1 we *engineered* features by hand. A CNN **learns** them — and the deeper the network, the more abstract the features become.

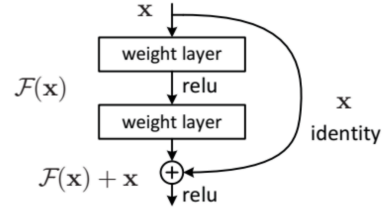
Example — skip connections

Depth is not free: **plain** very deep CNNs got *worse* training error. **ResNet** (He et al., 2015) fixed this by making each block learn a **residual** $F(x)$ and adding the input back unchanged:

$$y = F(x) + x$$

The **identity skip connection** carries the signal — and the gradient — straight through, so 50–150-layer networks become trainable. The *same* trick reappears in every transformer block, a few slides later.

(He et al., Deep Residual Learning for Image Recognition, 2015 — arXiv:1512.03385)



Part 2 — Transformers

Why sequences need attention

A CNN assumes **locality**; but in a sentence, the word that resolves *"it"* may be many tokens away. Earlier models (RNNs) read the sequence one step at a time — **serial**, slow, and forgetful over long ranges.

The **transformer** (Vaswani et al., *Attention is all you need*, 2017) throws out recurrence: every token can look at **every other token at once**, in parallel. The mechanism is **self-attention**.

Self-attention — queries, keys, values

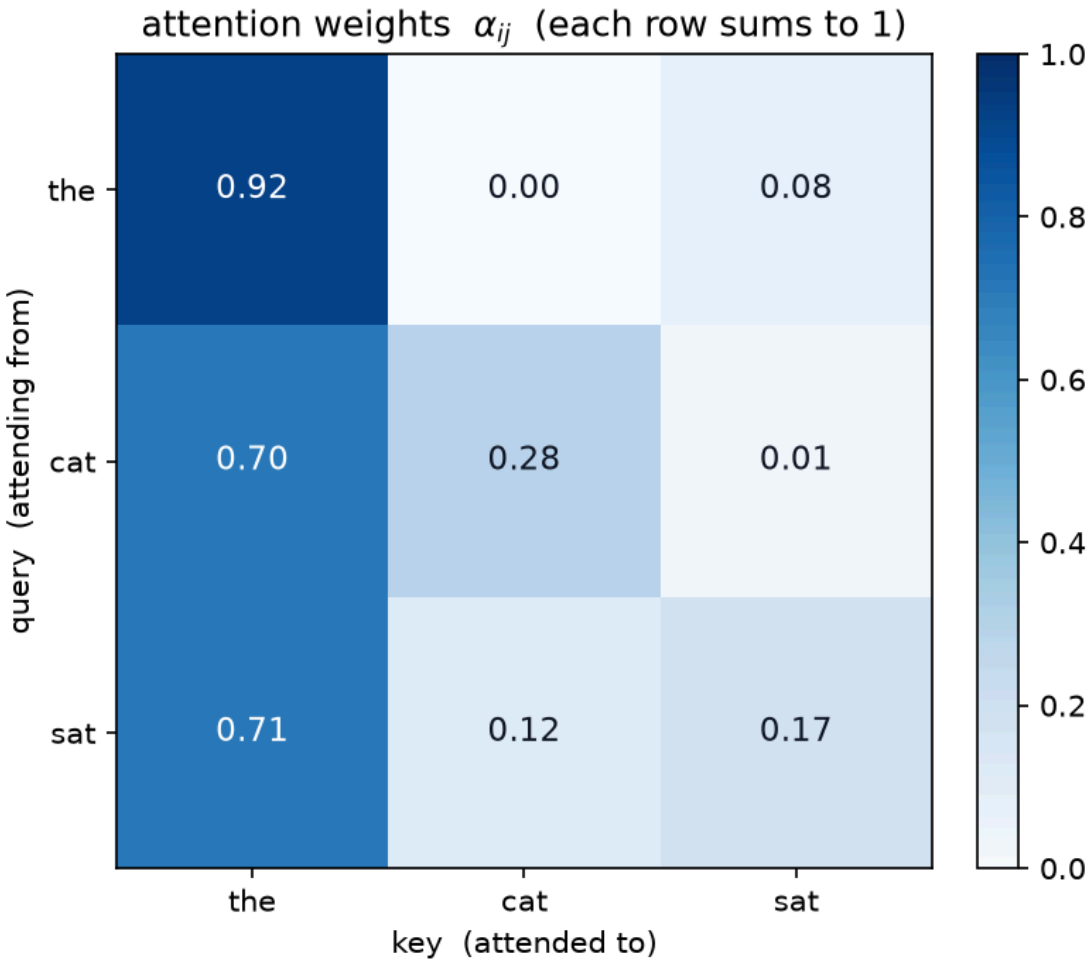
Each token is first turned into a vector (an **embedding**). For every token we then compute three projections:

- a **query** q — *"what am I looking for?"*,
- a **key** k — *"what do I offer?"*,
- a **value** v — *"what do I pass on if attended to?"*.

The attention weight of token i on token j is the **similarity** of q_i and k_j, turned into a distribution with softmax:

$$\alpha_{ij} = \mathrm{softmax}_j\left(\frac{q_i \cdot k_j}{\sqrt{d}}\right), \qquad \mathrm{out}_i = \sum_j \alpha_{ij}\, v_j$$

Each token's output is a **weighted mixture** of the values of the tokens it deems relevant. The next slide computes this by hand.


attention weights α_{ij} (each row sums to 1)
the
cat
sat
query (attending from)
0.92
0.00
0.08
0.70
0.28
0.01
0.71
0.12
0.17
the
cat
sat
key (attended to)
1.0
0.8
0.6
0.4
0.2
0.0

Multi-head attention & positional encoding

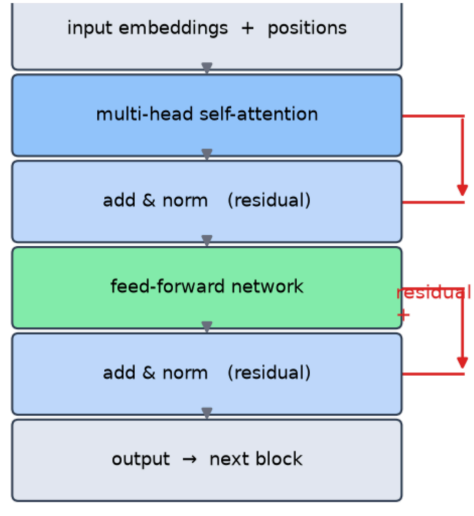
- **Multi-head attention** runs several attention "heads" in parallel, each with its own W_q, W_k, W_v. Different heads track different relationships (syntax, coreference, position); their outputs are concatenated.
- Attention is **permutation-invariant** — it has no built-in notion of order. **Positional encodings** (added to the embeddings) inject position, so *"dog bites man" ≠ "man bites dog"*.
- Stack these blocks over huge datasets and you get **BERT, GPT and ViT** — text, and increasingly vision and time series.

The transformer block

One block = **multi-head self-attention** + a small **feed-forward network**, each wrapped with a **residual connection** and **layer normalisation**:

- **residual** ($x + f(x)$) lets gradients flow through very deep networks — the same trick that made deep CNNs trainable;
- **layer norm** keeps activations well-scaled;
- the **feed-forward** layer gives every position its own non-linear transform.

Stack N blocks (dozens in an LLM), train with backpropagation and **Adam** — it is the *same* machinery as the MLP, scaled up.



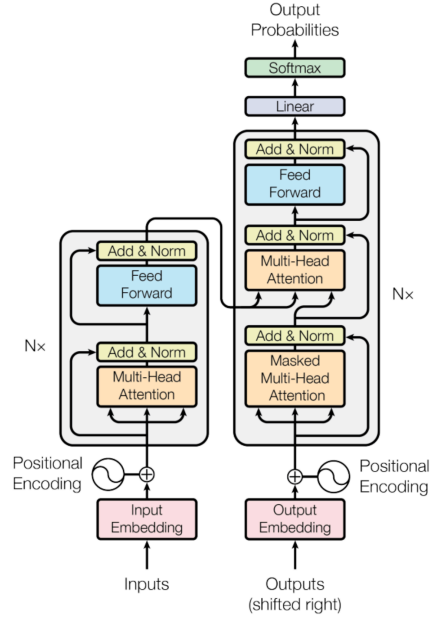
Example — the transformer, end to end

Zooming out from a single block: the original transformer (Vaswani et al., 2017) stacks an **encoder** and a **decoder**, each $N = 6$ of the blocks we just drew.

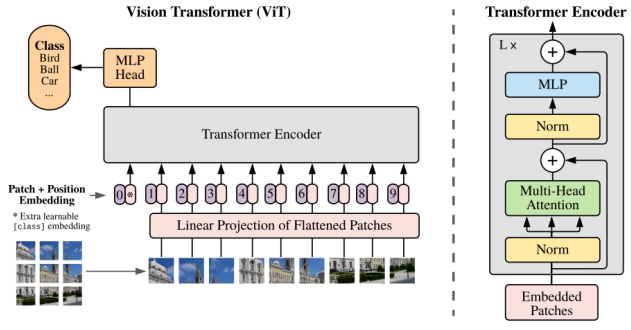
- the **encoder** (left) reads the input and builds contextual representations;
- the **decoder** (right) attends to the encoder's output and generates one token at a time;
- **residual connections** and **layer norm** wrap every sub-layer — exactly the $x + f(x)$ idea from the ResNet slide.

Modern LLMs keep the **decoder** stack; ViT keeps the **encoder**.

(Vaswani et al., Attention Is All You Need, 2017 — arXiv:1706.03762)



Example — the same recipe for images (ViT)



ViT (Dosovitskiy et al., 2021) shows the transformer is **not text-specific**: split an image into fixed-size **patches**, embed each patch as a **token**, and run the *same* encoder you just saw.

- a **patch plays the role of a word**; positional embeddings keep the 2-D layout;
- a learnable `[class]` token collects the image representation;
- with enough pretraining data, this matches or beats CNNs — and **unifies** vision and language under one architecture.

(Dosovitskiy et al., An Image is Worth 16×16 Words, 2021 — arXiv:2010.11929)

Part 3 — Learning without labels

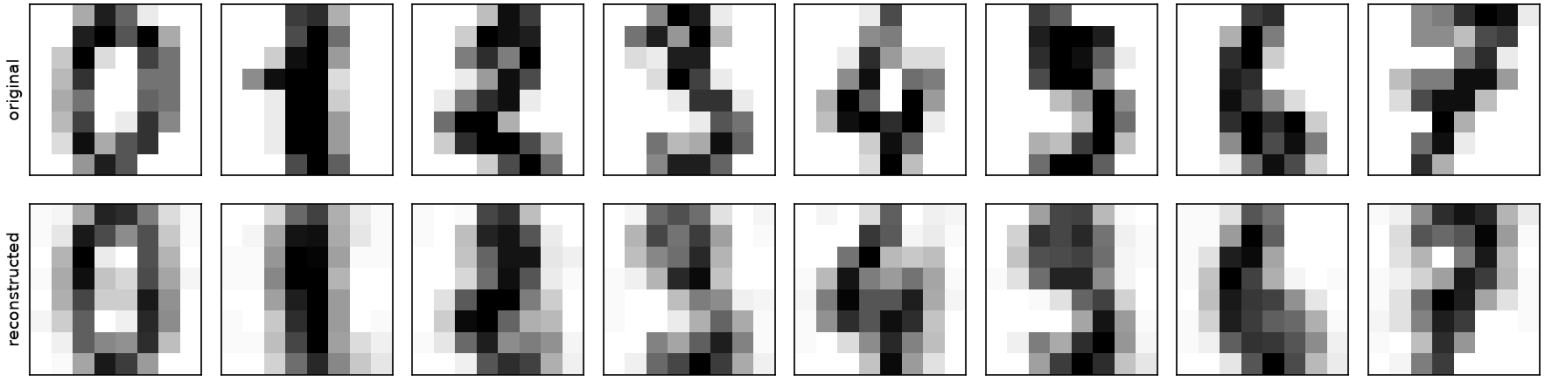
Autoencoders — representation learning

An **autoencoder** is a network trained to reconstruct its own input through a narrow **bottleneck**:

$$x \;\rightarrow\; \text{encoder} \;\rightarrow\; z \;\rightarrow\; \text{decoder} \;\rightarrow\; \hat{x}$$

Because the bottleneck is small, the network must learn a compact **representation** z that keeps the important structure. **No labels are needed** — this is **representation learning**. Autoencoders are used for compression, denoising and **pretraining** (the bridge to foundation models, next session).

A 64 → 32 → **8** → 32 → 64 autoencoder on handwritten digits, trained with `MLPRegressor` on its own input: 64 pixels squeezed through **8** numbers and reconstructed well enough to recognise.



The three model families of this course

	logistic regression	**trees / forests**	**neural networks**
boundary	straight line	axis-aligned steps	smooth, arbitrary
interpretability	high (weights)	tree = flowchart	low
feature scaling	recommended	not needed	required
training cost	seconds	seconds–minutes	minutes–hours
best at	fast baselines, calibrated probabilities	tabular data	images, text, signals

Rule of thumb: start with **logistic regression**, try a **random forest**, reach for a **neural network** when the data has *structure* (pixels, sequences) or the patterns are truly smooth — and reach for **CNNs / transformers** when that structure is rich.

Summary

- A flat **MLP ignores structure**; **CNNs** bake in locality, weight sharing and pooling — and learn their own features.
- **Transformers** replace recurrence with **self-attention**: every token attends to every other, in parallel; **multi-head** attention and **positional encodings** complete the picture.
- Both are trained with ordinary **backpropagation + Adam** — the machinery from Session 2, just stacked deep.
- **Autoencoders** learn representations **without labels**.

Next session: **foundation models** — what happens when we train these architectures on internet-scale data.